**College of Computing**

**Computer Science Department**

**Project Event Management CSC382**

**- Student Information**



| | | | |
|---|---|---|---|
| **Student 1:** | **Salem Hamdan Salem Al-Omrani** | **ID:** | **202322090** |
| **Student 2:** | **Abdullah Ali Khalaf Al-Rashidi** | **ID:** | **202422065** |
| **Student 3:** | **Ayman Abdullah Al-Omari** | **ID:** | **202412014** |



# Phase 4 – System Design Document

This document details the architectural and structural design of the Event Management System, adhering to academic standards and formal technical writing conventions. The design is derived from the analysis of the provided project files (EventManagementCSC382).

## 1. System Architecture

The Event Management System employs a conventional web application architecture, primarily based on the **Client-Server** model and structured according to the **Three-Tier Architecture** pattern.

### 1.1. Architectural Pattern: Three-Tier Architecture

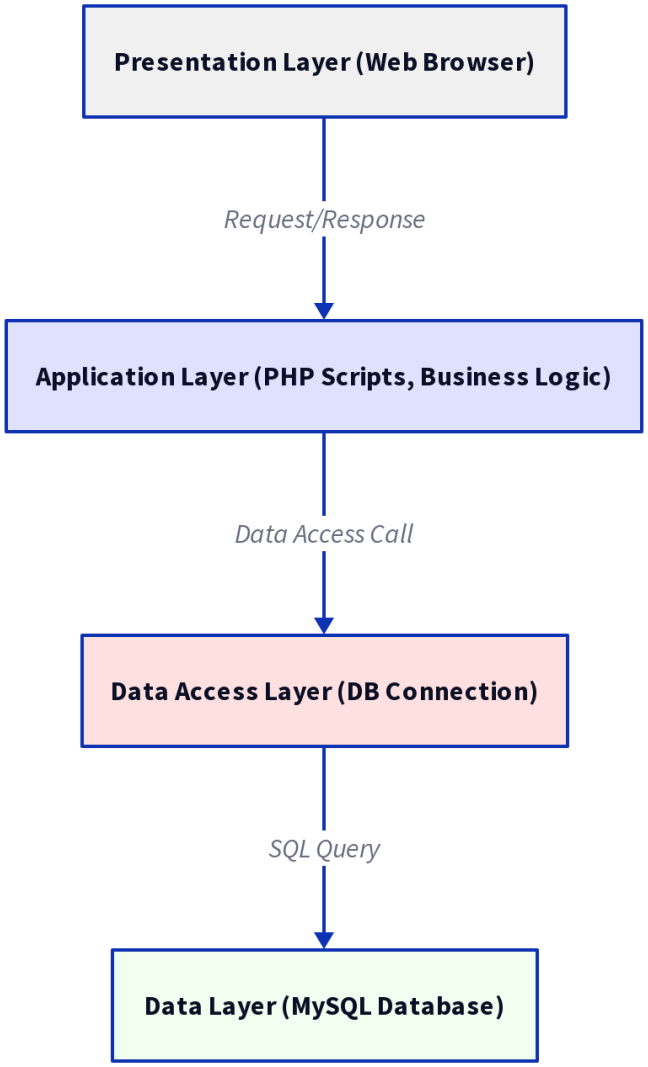
The system is logically separated into three distinct tiers, promoting modularity, scalability, and maintainability.

| Tier | Component | Technology/Role |
|---|---|---|
| **Presentation Tier** | Client-Side Interface | HTML, CSS, JavaScript. Responsible for rendering the user interface and handling user interaction. |

| | | |
|---|---|---|
| **Application Tier** | Web Server & Business Logic | PHP scripts (index.php, pages/*.php, includes/*.php). This tier processes client requests, executes business logic (e.g., authentication, event validation), and interacts with the Data Tier. |
| **Data Tier** | Database Management System | MySQL. Responsible for persistent storage of all system data, including user profiles, event details, and registrations. |

## 1.2. Layered Diagram (Conceptual)

The system can be conceptually viewed as a layered structure, where each layer provides services to the layer above it.




## 1.3. Architectural Style: Client-Server and MVC Considerations

The core style is **Client-Server**, where the client (web browser) requests resources from the server (PHP application).

While the project structure (PHP files mixing presentation and logic) does not strictly enforce a modern **Model-View-Controller (MVC)** framework, the separation of concerns is partially achieved:

- **Model**: The database structure and the data access functions (db_connect.php, SQL queries).
- **View**: The HTML structure within the PHP files (header.php, footer.php, pages/*.php).
- **Controller**: The main logic in the PHP files that handles routing, input validation, and data manipulation (e.g., index.php, login.php).

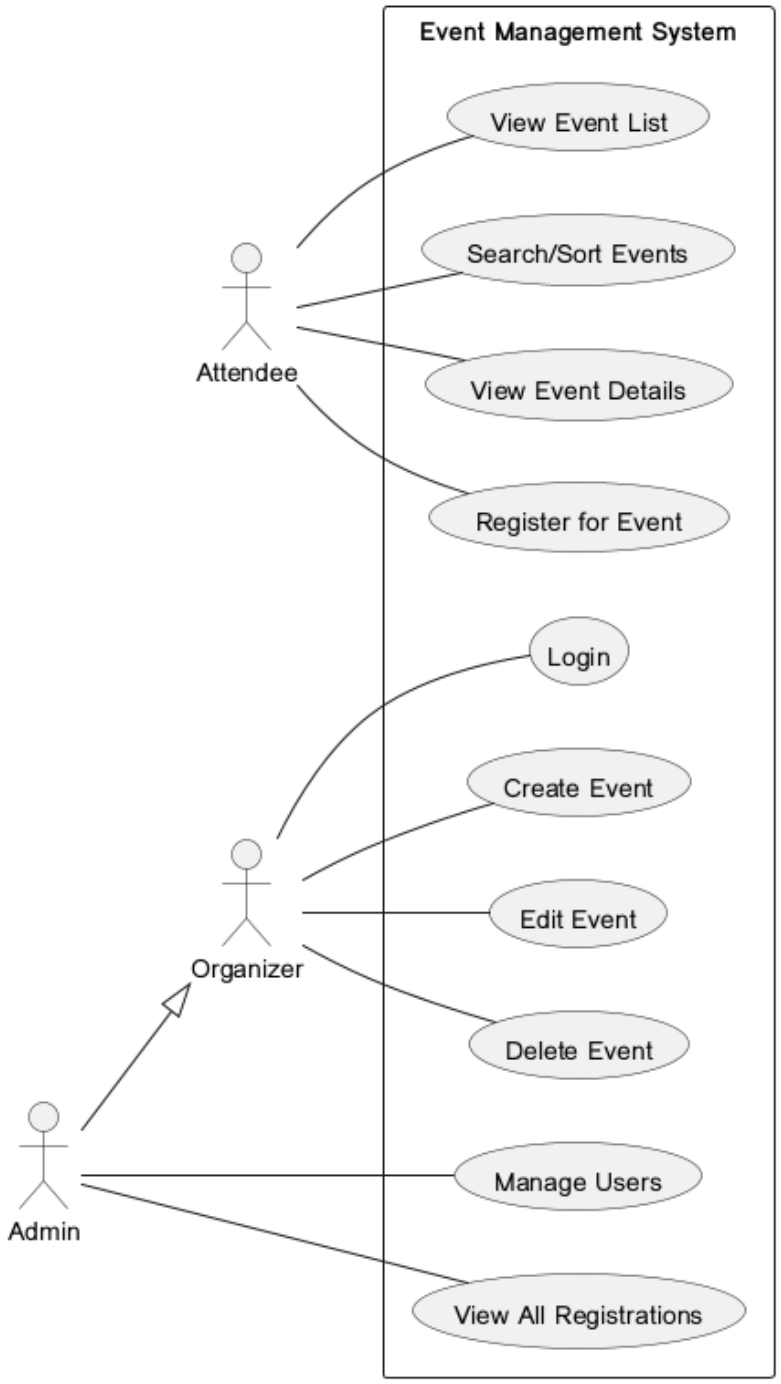
# 2. UML Models

The following Unified Modeling Language (UML) diagrams illustrate the system's structure, behavior, and deployment environment.

## 2.1. Use Case Diagram

The Use Case Diagram identifies the primary actors and their interactions with the system's main functionalities.

| Actor | Description | Key Use Cases |
|---|---|---|
| **Attendee** | Unauthenticated or basic authenticated user. | View Event List, Search/Sort Events, View Event Details, Register for Event. |
| **Organizer** | Authenticated user with permission to manage events. | Login, Create Event, Edit Event, Delete Event. |
| **Admin** | Authenticated user with full system privileges. | All Organizer Use Cases, Manage Users, View All Registrations. |




## 2.2. Class Diagram

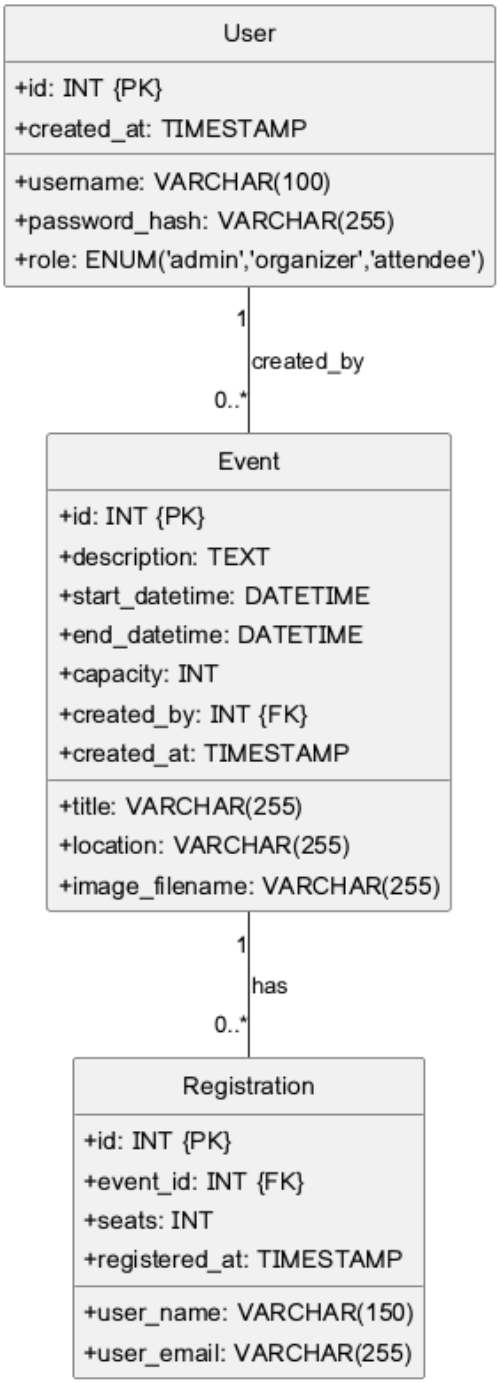
The Class Diagram illustrates the static structure of the system, showing the main classes, their attributes, and the relationships between them, based on the database schema.

| **Class** | **Key Attributes** |
|---|---|
| **User** | id (PK), username, password_hash, role (admin, organizer, attendee), created_at. |
| **Event** | id (PK), title, description, location, start_datetime, capacity, image_filename, created_by (FK to User). |

| | |
|---|---|
| **Registration** | id (PK), event_id (FK to Event), user_name, user_email, seats, registered_at. |

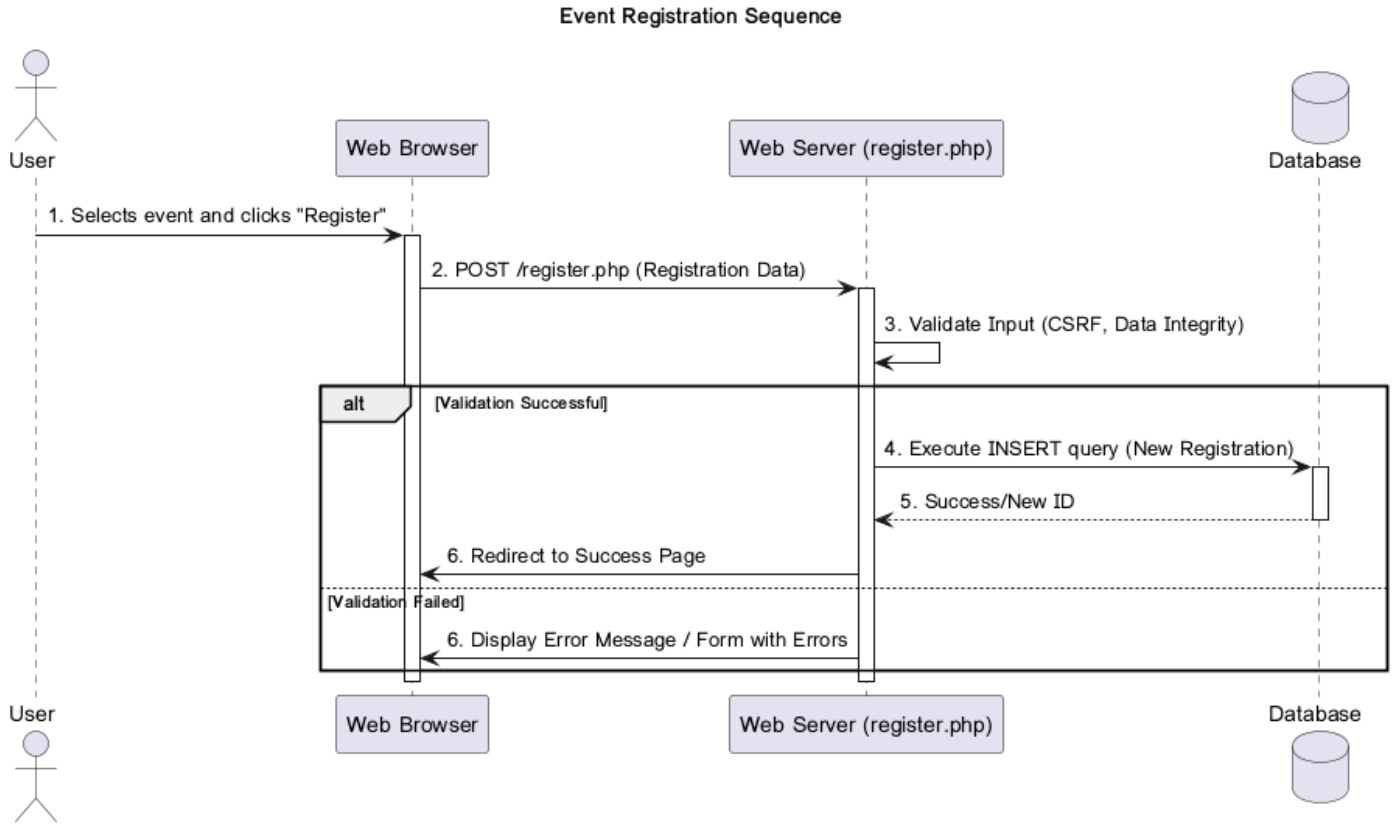
**Relationships:**

- **User** $\xrightarrow{1..*}$ **Event**: A User (Organizer/Admin) creates multiple Events.
- **Event** $\xrightarrow{1..*}$ **Registration**: An Event can have multiple Registrations.



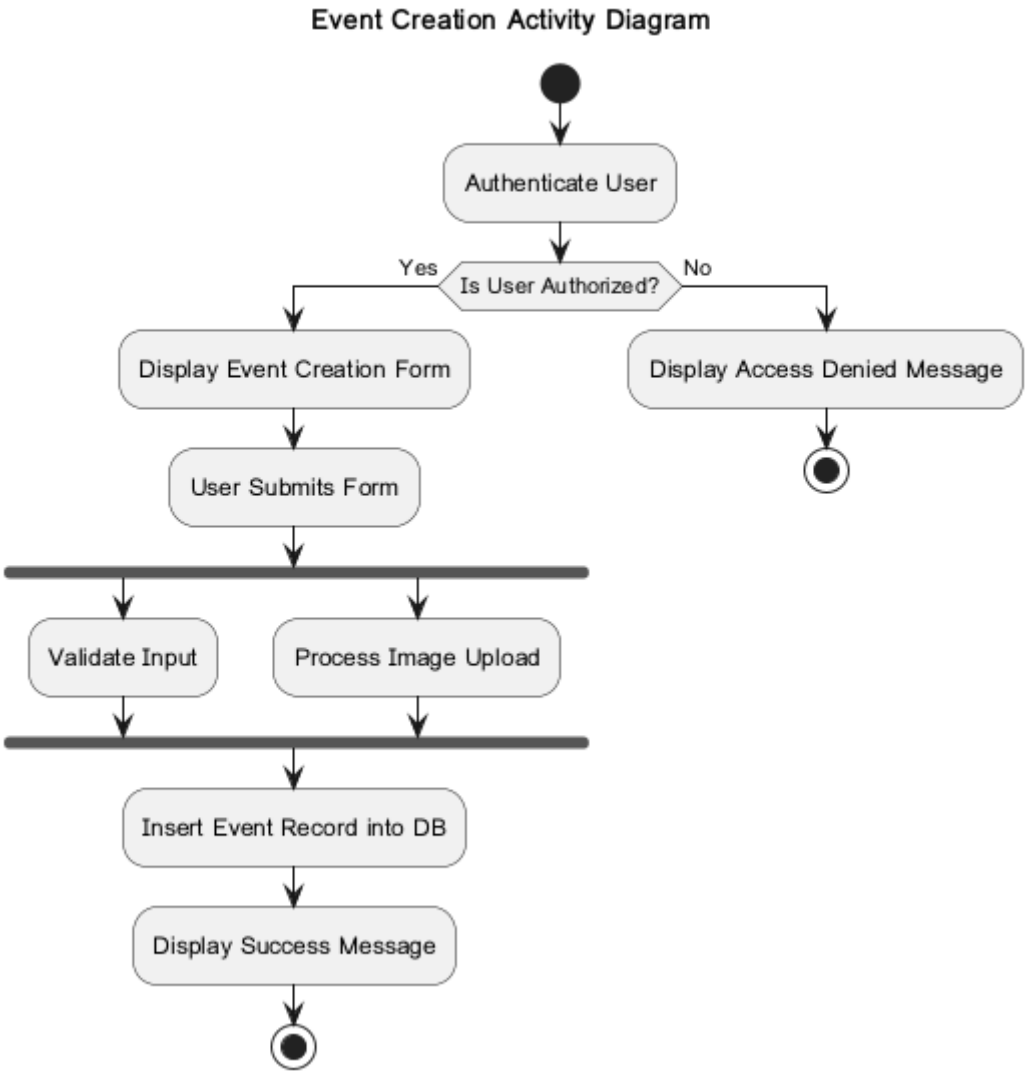

## 2.3. Sequence Diagram (Event Registration)

This diagram models the dynamic interaction flow for the critical use case of an Attendee registering for an event.




## 2.4. Activity Diagram (Event Creation)

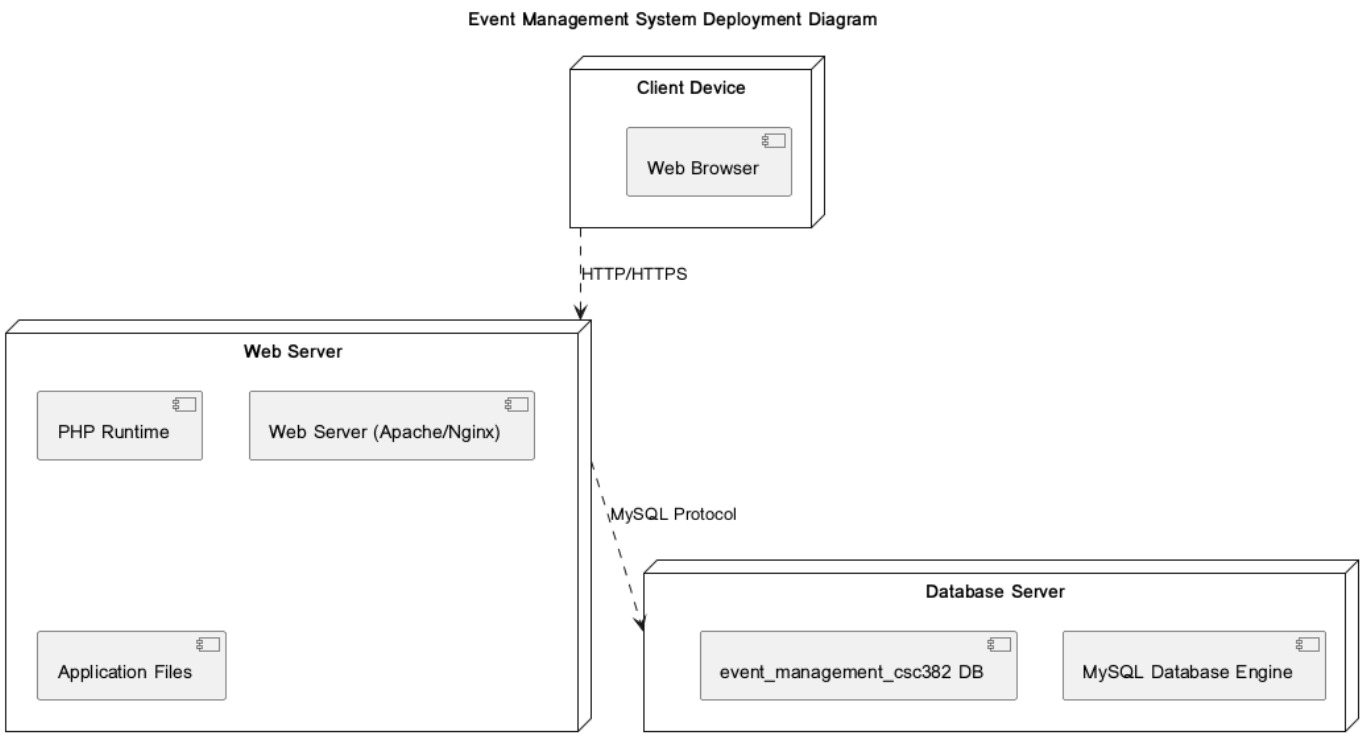
The Activity Diagram illustrates the workflow for an Organizer or Admin to create a new event, including decision points and parallel processes.




## 2.5. Deployment Diagram

The Deployment Diagram illustrates the physical deployment of the software components onto the hardware nodes.

| **Node** | **Description** | **Artifacts/Components** |
|---|---|---|
| **Client Device** | User's machine (e.g., Desktop, Mobile). | Web Browser (Chrome, Firefox, etc.) |
| **Web Server** | Server hosting the application logic. | PHP Runtime, Web Server (Apache/Nginx), Event Management Application Files. |
| **Database Server** | Dedicated server for data persistence. | MySQL Database Engine, event_management_csc382 Database. |

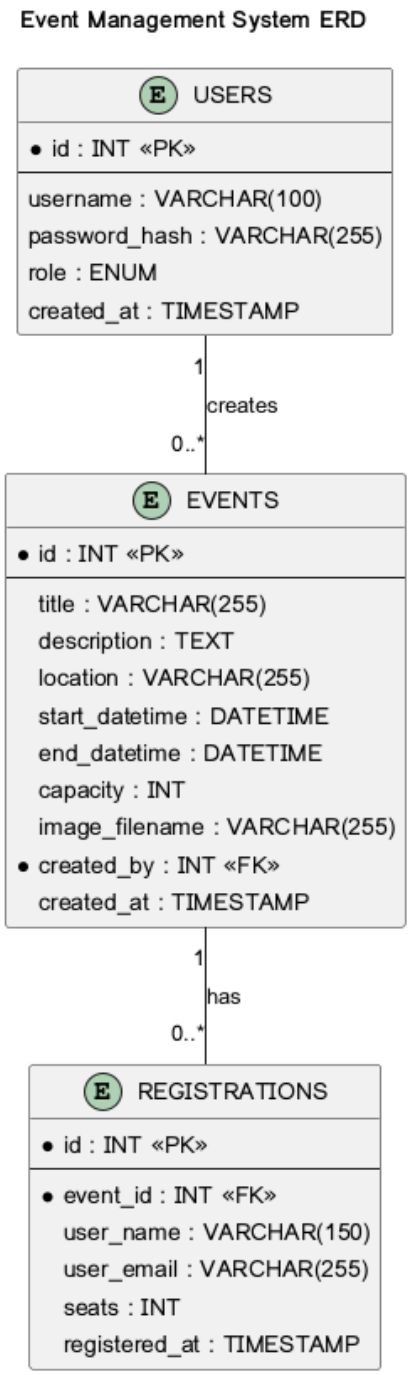



# 3. Database Design

The database design is based on the provided event_management.sql schema, ensuring data integrity and efficient retrieval.

## 3.1. Entity-Relationship Diagram (ERD)

The ERD visually represents the entities and their relationships, forming the backbone of the Data Tier.




## 3.2. Tables, Primary Keys, and Relationships

The following tables define the structure of the database.

| Table Name | Primary Key (PK) | Key Columns | Relationships |
|---|---|---|---|
| **users** | id | username, password_hash, role | PK for **users** |

| | | | |
|---|---|---|---|
| **events** | id | title, start_datetime, created_by | FK: created_by references users.id (One-to-Many) |
| **registrations** | id | event_id, user_name, user_email | FK: event_id references events.id (One-to-Many) |

**Table Details:**

| Table | Column Name | Data Type | Constraint | Description |
|---|---|---|---|---|
| **users** | id | INT | PK, AUTO_INCREMENT | Unique identifier for the user. |
| | username | VARCHAR(100) | UNIQUE, NOT NULL | User's login name. |
| | password_hash | VARCHAR(255) | NOT NULL | Hashed password for security. |
| | role | ENUM | NOT NULL, DEFAULT 'attendee' | User role: 'admin', 'organizer', 'attendee'. |
| **events** | id | INT | PK, AUTO_INCREMENT | Unique identifier for the event. |
| | title | VARCHAR(255) | NOT NULL | Title of the event. |
| | description | TEXT | | Detailed description. |
| | start_datetime | DATETIME | NOT NULL | Start date and time of the event. |
| | created_by | INT | FK | ID of the user who created the event. |
| **registrations** | id | INT | PK, AUTO_INCREMENT | Unique identifier for the registration record. |
| | event_id | INT | FK, NOT NULL | ID of the event being registered for. |

| | | | | |
|---|---|---|---|---|
| | user_email | VARCHAR(255) | NOT NULL | Email of the person registering. |
| | seats | INT | DEFAULT 1 | Number of seats/tickets requested. |

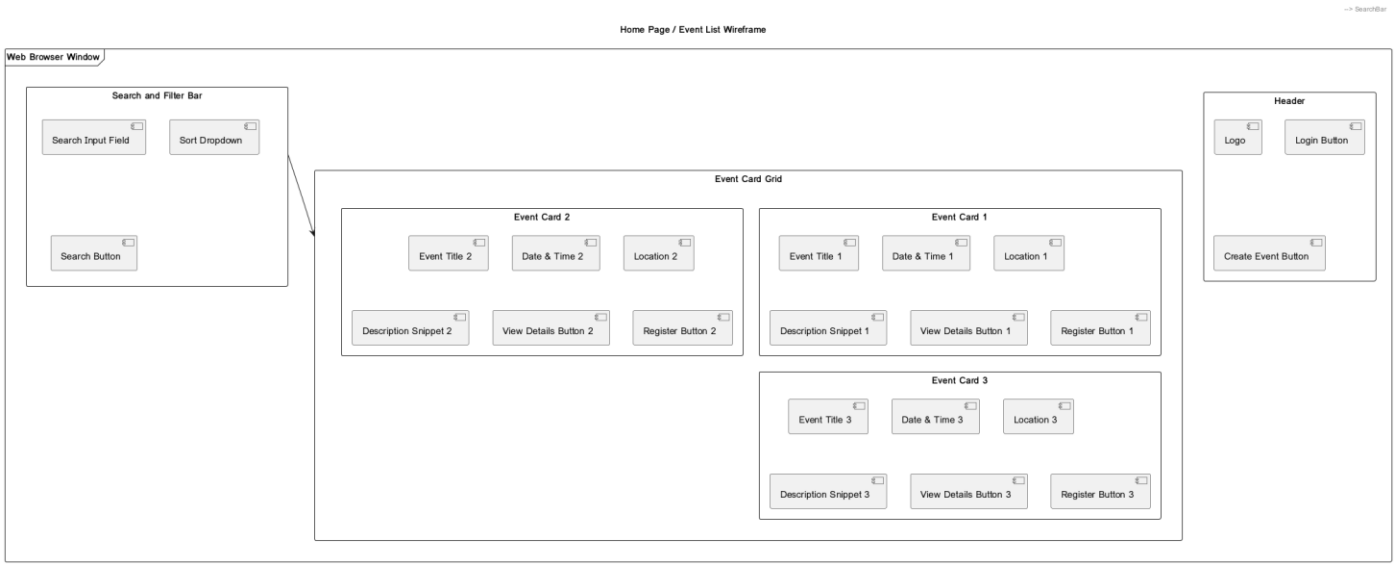
# 4. UI Mockups / Wireframes

The user interface (UI) design focuses on clarity, ease of navigation, and responsiveness, catering to both attendees viewing events and organizers managing them.

## 4.1. Home Page / Event List

This is the main landing page, accessible to all users.

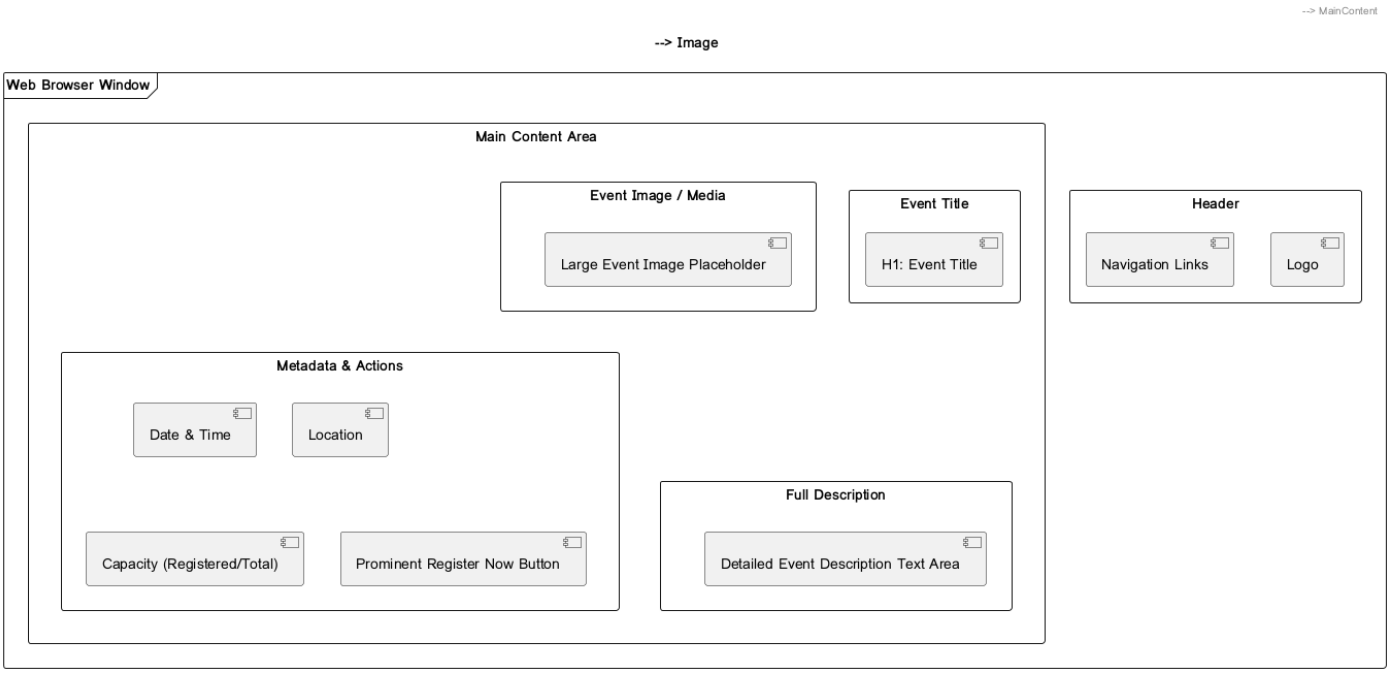
**Description:** A clean, responsive layout featuring a prominent header and a main content area. The header includes the system title and navigation links for "Login" and "Create Event" (if authorized). The main content displays a grid or list of upcoming events. Each event card shows the title, date, location, a brief description snippet, and buttons for "View Details" and "Register." A search bar and a sort dropdown (by date, title, or creation date) are positioned above the list.




## 4.2. Event Details Page

This page provides comprehensive information about a single event.

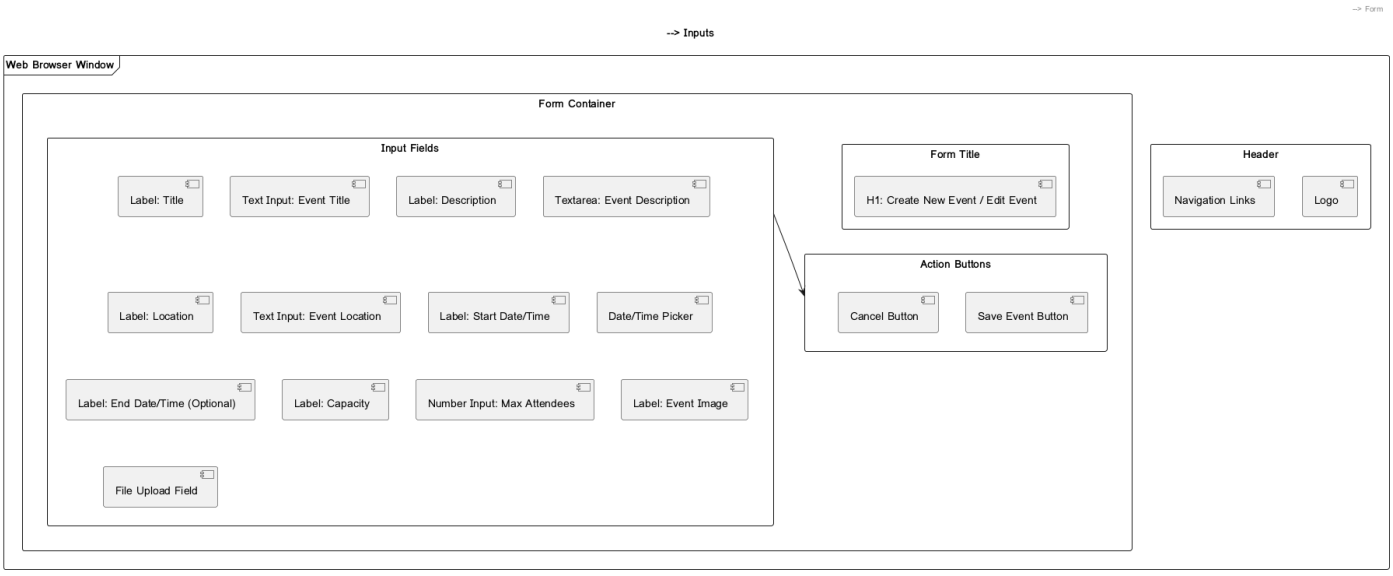
**Description:** The page features the event title prominently. Below the title, a large image (if available) is displayed. Key details are presented in a sidebar or metadata section: date, time, location, capacity (with current registration count). The main area contains the full, detailed description of the event. A clear "Register Now" button is placed at the top and bottom of the description.




## 4.3. Event Creation / Edit Form

This form is restricted to authenticated Organizers and Admins.

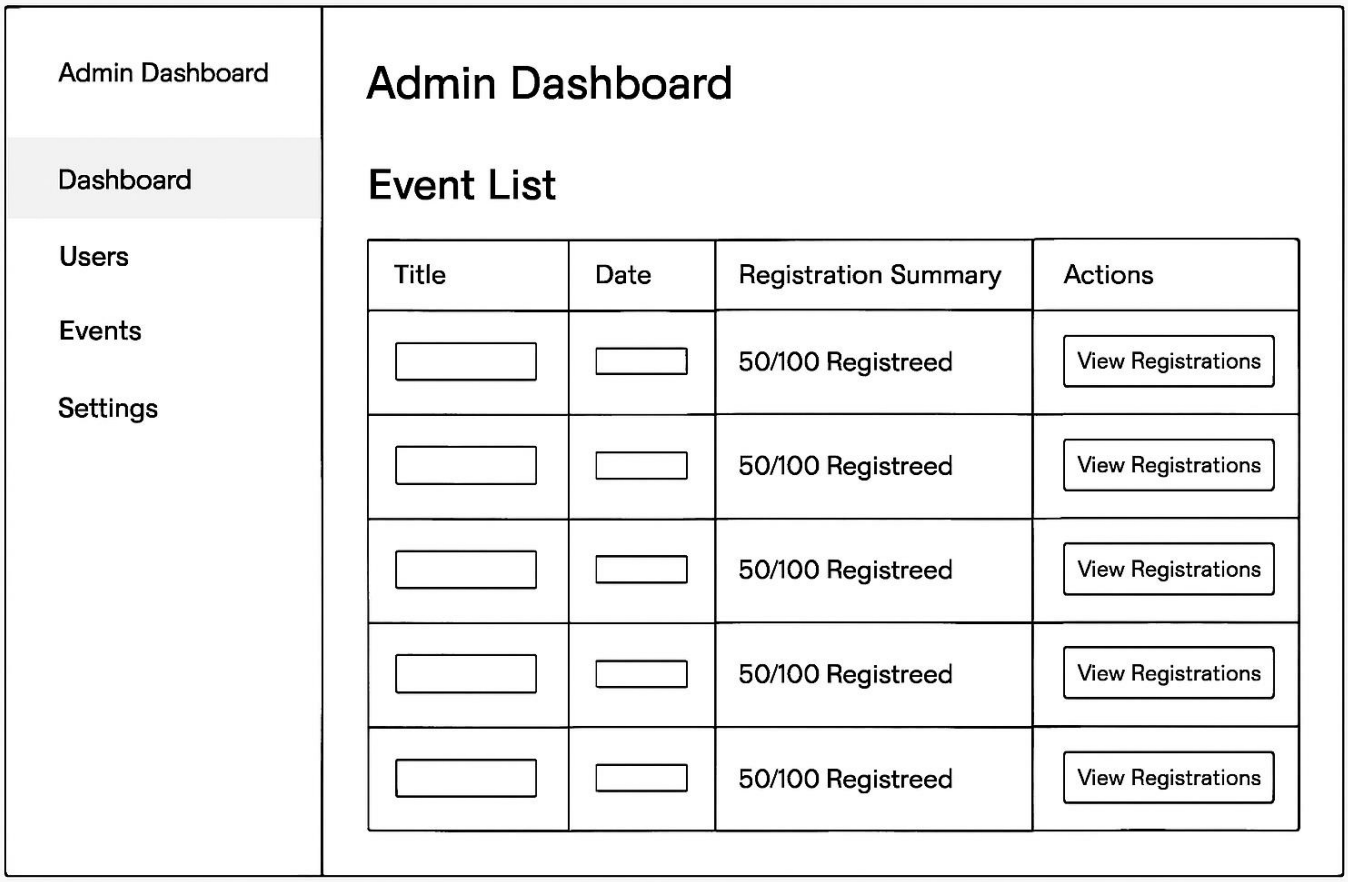
**Description:** A multi-field form designed for creating or modifying an event. Fields include: **Title** (text input), **Description** (textarea), **Location** (text input), **Start Date/Time** (datetime picker), **End Date/Time** (optional datetime picker), **Capacity** (number input), and **Event Image** (file upload field). Clear validation messages are displayed for required fields. The form concludes with a "Save Event" or "Update Event" button.

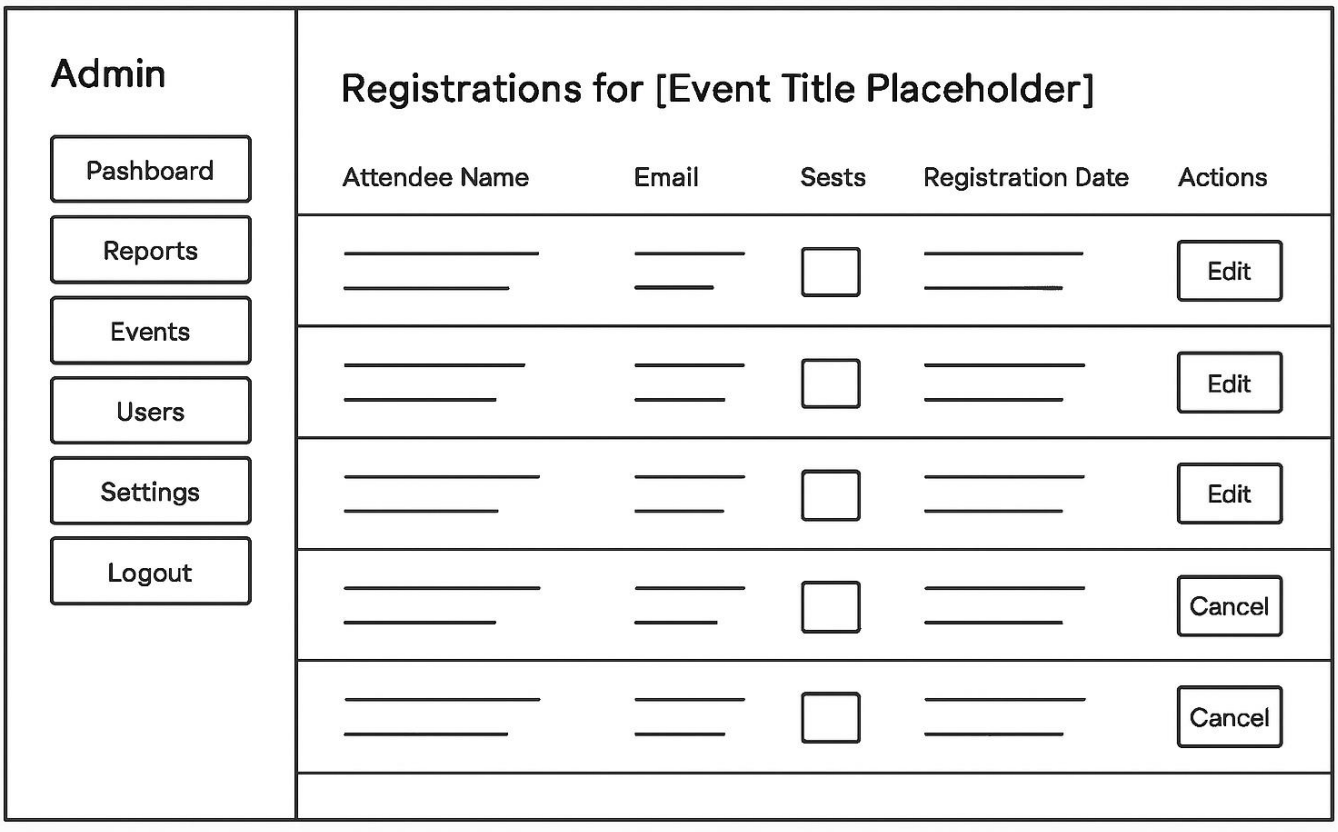


## 4.4. Admin Dashboard (Registration Management)

This page is restricted to Admin users for system oversight.

**Description:** A simple dashboard view focused on managing registrations. It displays a table listing all events. For each event, it shows the title, date, and a summary of registrations (e.g., "50 registered / 100 capacity"). A "View Registrations" button for each event leads to a detailed table showing all attendees for that specific event (Attendee Name, Email, Seats, Registration Date).


Admin Dashboard
Dashboard
Users
Events
Settings
Admin Dashboard
Event List
Title
Date
Registration Summary
Actions
50/100 Registreed
View Registrations
50/100 Registreed
View Registrations
50/100 Registreed
View Registrations
50/100 Registreed
View Registrations
50/100 Registreed
View Registrations


Admin
Pashboard
Reports
Events
Users
Settings
Logout
Registrations for [Event Title Placeholder]
Attendee Name
Email
Sests
Registration Date
Actions
Edit
Edit
Edit
Cancel
Cancel